
DATOS Y PANDAS

Fernanda Sobrino

2026

¿Qué son los datos?

Los datos registran eventos

- ▶ Idealmente los datos representan eventos relacionados con entidades: cosas que les pasan o que hacen
- ▶ Un renglón de datos dice qué de quién, en dónde y cuándo
- ▶ Casi toda interacción en el mundo genera datos; el reto no es tenerlos, es poder analizarlos
- ▶ En política pública: un trámite, una inspección, un pago, una consulta médica

Estructurados y no estructurados

	Estructurados	No estructurados
Qué son	Tablas: números, fechas, categorías	Texto, audio, imagen, video
Origen	Transacciones, sensores, formularios	Correos, documentos, redes
Formato	CSV, Excel, SQL	PDF, mp3, mp4, NoSQL

- ▶ Hay un tercer tipo, semi-estructurados (JSON, XML): tienen estructura, pero no una tabla fija
- ▶ La mayoría de los datos del mundo son no estructurados — y casi siempre hay que convertirlos en tabla para poder usarlos

Una hoja de Excel no siempre es una tabla

- ▶ Datos estructurados: filas, columnas, y una sola cosa por celda
- ▶ No cuentan como tabla si tienen:
 - ▷ celdas combinadas o títulos a media hoja
 - ▷ totales mezclados con observaciones
 - ▷ gráficas, tablas dinámicas, colores como información
- ▶ Regla práctica: si para leer el archivo hay que “saber dónde está la información”, todavía no son datos

Los retos de una base real

ID	Nombre	CP	Ciudad	Estado	Salario
1	Green	60610	Chicago	IL	30k
2	Green	60611	Chicago	IL	32k
3	Peter		New Yrk	NY	40k
6	Chuck	90057	San Francisco	CA	30k

- ▶ ¿Los dos “Green” son duplicados o dos personas?
- ▶ “New Yrk”: error de captura — ¿cuántas categorías tiene realmente Ciudad?
- ▶ CP faltante: valor faltante
- ▶ CP 90057 con ciudad San Francisco: error de captura (ese CP es de Los Ángeles)

Anatomía de una tabla

Renglones y columnas

- ▶ Un renglón = una observación. Lo primero que hay que poder decir de una base es “cada renglón es un ____”
 - ▷ un vuelo, un avión, una hora de clima, un municipio-año
 - ▷ si no puedes terminar la frase, no puedes verificar nada que salga de ahí
- ▶ Una columna = una variable, con un tipo
 - ▷ numéricas (retraso en minutos, distancia)
 - ▷ categóricas (aerolínea, destino)
 - ▷ fechas y horas (que pandas lee como texto si no le dices)
- ▶ El tipo importa: una variable numérica o fecha guardada como texto no se va a poder usar

La llave primaria

- ▶ Llave primaria: la columna (o combinación) que identifica de forma única cada renglón de su propia tabla
- ▶ Llave foránea: identifica de forma única una observación en otra tabla
- ▶ Ejemplos en `flights`:
 - ▷ `planes`: la llave es `plane` (matrícula del avión)
 - ▷ `weather`: la llave es `date` + `hour` — es una llave compuesta
- ▶ La mayoría de las bases no te dicen cuál es su llave: hay que encontrarla

Encontrar (y verificar) la llave primaria

```
# ¿'plane' identifica un solo renglón en planes?  
planes.groupby('plane').size().loc[lambda s: s > 1]  
# vacío => sí es llave primaria
```

- ▶ Si sale vacío, plane es llave primaria
- ▶ Si no sale vacío, tienes duplicados o la llave es compuesta
- ▶ ¿Y si no hay llave? Se puede crear una (el número de renglón), pero entonces **no puedes unir** con nadie más

pandas

pandas y el DataFrame

- ▶ pandas es la biblioteca de Python para datos tabulares
- ▶ Su objeto central es el **DataFrame**: una tabla bidimensional con columnas de distinto tipo
- ▶ Todo lo que sigue son seis verbos: **seleccionar, filtrar, transformar, mutar, agrupar y unir**
- ▶ Alrededor viven numpy (matrices), matplotlib (gráficas), sklearn (modelos)
- ▶ Se aprende haciendo ejercicios, no leyendo documentación

Este material está adaptado del curso de dplyr de Hadley Wickham.

Cómo trabajaremos: estas slides son el mapa

- ▶ **Estas slides son el resumen conceptual**; todo el código se ve y se corre en el notebook
- ▶ Abran ahora: `notebooks/Pandas.ipynb` (datos: vuelos que salieron de Houston en 2011 — `flights`, `weather`, `planes`, `airports`)
- ▶ El ritmo de la clase: una idea aquí → al notebook a hacerla → de regreso solo para el siguiente concepto
- ▶ Para estudiar: el notebook es la referencia completa; estas slides son el repaso de que hace el notebook

El mapa de los seis verbos

Verbo	En pandas	Responde
Seleccionar	<code>.loc / .iloc / []</code>	¿qué renglones y columnas veo?
Filtrar	máscaras, <code>query</code>	¿qué observaciones cumplen la condición?
Mutar	<code>df['nueva'] = ...</code>	¿qué variable me falta construir?
Ordenar	<code>.sort_values()</code>	¿quién está arriba y quién abajo?
Agrupar	<code>.groupby().agg()</code>	¿cómo se compara entre grupos?
Unir	<code>.merge()</code> (<i>próxima clase</i>)	¿qué sabe otra tabla de mis renglones?

Tres trampas (las veremos en vivo en el notebook)

1. **Los faltantes son contagiosos:** `np.nan` no es igual a nada — ni a sí mismo. Un filtro nunca devuelve los renglones con faltante, así que `filtrar(x > 60) + filtrar(x <= 60)` **no** suman el total de la base
 2. **Las agregaciones descartan NaN en silencio:** `mean()` es el promedio *de los que sí tienen dato*, no de todos
 3. **Cada fuente inventa su código de faltante** (-99, "N/D", " "): si no lo declaras al leer, entra como número y contamina todos los promedios
- ▶ Regla transversal: pandas nunca modifica en su lugar — casi todas las operaciones devuelven una copia nueva del DataFrame en lugar de cambiar el original.
 - ▶ Si no asignaste el resultado, solo te lo va a mostrar no se va a guardar en ningún lado.

Notebook

Lo que vamos a hacer en Pandas.ipynb

1. **Leer** los datos y hacer el reconocimiento inicial (`head`, `shape`, `info`, `dtypes`) — siempre antes de cualquier análisis
2. **Índices y selección**: `.loc` (etiquetas) vs `.iloc` (posiciones)
3. **Filtrar**: máscaras booleanas, `query`, y qué pasa con los faltantes
4. **Mutar y ordenar**: columnas nuevas a partir de otras
5. **Agrupar**: dividir–aplicar–combinar con `groupby` y `agg`
6. **Encadenar** todo con pipes

Qué verificar si le pides a un agente o un LLM que lo haga

- ▶ **Qué te genere código** NO aceptes que el agente solo te de resultados, exígele que genere el código para que lo puedas leer, correr y cambiar.
- ▶ **NUNCA** aceptes que los agentes hagan las cosas on the fly
- ▶ **Cuenta renglones y columnas** después de leer: `df.shape` contra lo que dice la fuente. ¿Leyó el archivo completo?
- ▶ **Revisa dtypes**: una columna numérica leída como `object` significa que hay un código de faltante (-99, "N/D") que nadie declaró

Qué verificar si le pides a un agente o un LLM que lo haga

- ▶ **Pregunta por la llave:** `df.duplicated(llave).sum() == 0`. Si el agente asumió que hay una llave única, verifícalo tú
- ▶ **Después de cada filtro**, cuenta cuántos renglones se fueron y por qué — los NaN desaparecen sin avisar y no reaparecen en el filtro complementario
- ▶ **Después de cada groupby**, revisa el tamaño de los grupos: un promedio sobre 3 observaciones no es un promedio
- ▶ **Reconstruye un número por otro camino:** `df['x'].sum()` debe coincidir con `df.groupby('g')['x'].sum().sum()`